Studi Kasus Analisis dan Implementasi Sistem Berbasis Objek

Program Pendaftaran Magang Mahasiswa

Disusun oleh:
Nama : Afdila Dwiyani
NPM : 250215001
Kelas : TRPL 1A

JURUSAN KOMPUTER DAN BISNIS
PROGRAM STUDI TEKNOLOGI REKAYASA PERANGKAT LUNAK POLITEKNIK NEGERI CILACAP
TAHUN 2026

A. Bagian 1 – Analisis Sistem

1. Identifikasi minimal 3 class utama yang terdapat pada sistem tersebut!

No	Class	Deskripsi
1.	Perusahaan	Sebagai Class entitas yang menyimpan informasi mengenai instansi atau tempat tujuan magang.
2.	Mahasiswa	Sebagai parent Class(Induk). Menyimpan data identitas dasar pengguna yang melakukan pendaftaran.
3.	Magang	Sebagai Class detail program. Mengelola spesifikasi kontrak magang dan status persetujuan.
4.	FormulirMagang	Sebagai Subclass(Class Turunan) dari Mahasiswa. Class ini berfungsi sebagai "perekat" yang menggabungkan semua data.
5.	Main	Sebagai Class penggerak (Driver Class) yang berisi logika interaksi pengguna dan penyimpanan data sementara.

2. Tentukan atribut setiap class!
   a. Class Mahasiswa (Induk)
      - nim: Menyimpan nomor induk mahasiswa sebagai identitas unik.
      - nama: Menyimpan nama lengkap mahasiswa
      - jurusan: Menyimpan bidang studi atau program studi mahasiswa
   b. Class Perusahaan (Entity)
      - id: Kode unik atau identitas dari perusahaan
      - nama: Nama resmi perusahaan tujuan
      - alamat: Lokasi atau alamat kantor perusahaan
   c. Class Magang (Detail Program)
      - idMagang: Kode referensi untuk posisi magang tertentu
      - posisi: Nama jabatan atau divisi yang dilamar
      - durasi: Masa waktu magang dalam satuan bulan
      - status: Kondisi pendaftaran saat ini
   d. Class FormulirMagang (Subclass)
      - perusahaanTujuan: Sebuah variabel referensi yang menampung objek dari class Perusahaan.
      - detailMagang: Sebuah variabel referensi yang menampung objek dari class Magang.
      - (Atribut Warisan): Secara otomatis memiliki akses ke atribut nim, nama, dan jurusan milik class Mahasiswa.

3. Tentukan method/perilaku dari setiap class!
   a. Class Mahasiswa
      - tampilkanData(): Berfungsi untuk mencetak data statis mahasiswa berupa NIM, Nama, dan Jurusan ke konsol.
   b. Class Perusahaan
      - tampilkanData(): Berfungsi untuk menampilkan informasi detail perusahaan yang mencakup

ID, Nama Perusahaan, dan Alamat kantor.

c. Class Magang
   - ubahStatusPendaftaran(statusNew): Method ini menerima parameter string untuk memperbarui kondisi pendaftaran (misalnya dari "Menunggu" menjadi "Diterima").
   - tampilkanInfo(): Berfungsi untuk menampilkan detail kontrak magang seperti ID magang, posisi yang dilamar, durasi, dan status saat ini.

d. SubClass FormulirMagang
   - tampilkanBuktiPendaftaran(): Method utama yang memicu pemanggilan method tampilkanData() dari parent (Mahasiswa) serta memanggil method dari objek Perusahaan dan Magang untuk menghasilkan satu laporan utuh (Kartu Pendaftaran).

4. Jelaskan hubungan antar class

a. Hubungan Pewarisan (Inheritance)

   Hubungan ini terjadi antara class Mahasiswa sebagai Parent Class (Induk) dan class FormulirMagang sebagai *Subclass* (Anak).

   - Karakteristik: Ditandai dengan simbol panah segitiga kosong pada diagram dan kata kunci extends pada kode Java.
   - Fungsi: Class FormulirMagang mewarisi seluruh atribut (nim, nama, jurusan) dan method (tampilkanData()) milik class Mahasiswa.
   - Implementasi: Dalam konstruktor FormulirMagang, digunakan perintah super(m.getNim, m.getNama, m.getJurusan) untuk menginisialisasi data yang berasal dari induknya.

b. Hubungan Komposisi (Composition)

   Hubungan antara FormulirPendaftaran sebagai kelas utama yang mengandung objek Perusahaan dan Magang.

   - Karakteristik: Disimbolkan dengan belah ketupat hitam (diamond). Hubungan bersifat sangat kuat; jika objek utama dihapus, objek di dalamnya juga ikut terhapus.
   - Fungsi: Menyatukan data tujuan perusahaan dan detail pekerjaan menjadi satu kesatuan formulir yang utuh.
   - Implementasi: Objek Perusahaan dan Magang dijadikan atribut di dalam kelas FormulirPendaftaran dan diisi melalui parameter konstruktor.

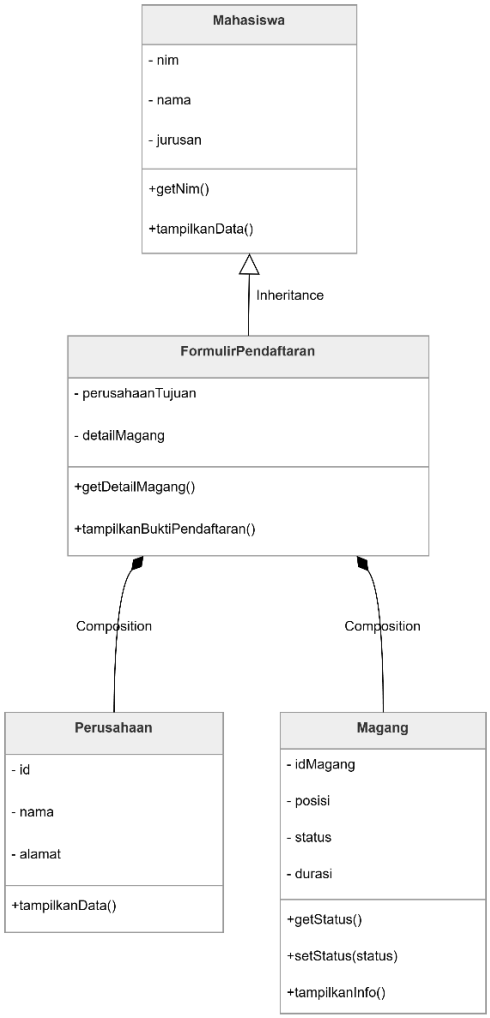
c. Karakteristik Encapsulation (Pembungkusan)

   Prinsip ini diterapkan pada seluruh class untuk mengatur hak akses terhadap data-data di dalamnya.

   - Karakteristik: Ditandai dengan simbol minus (-) pada diagram yang berarti atribut bersifat private. Data tidak dapat diakses atau diubah secara langsung dari luar class.
   - Fungsi: Melindungi integritas data agar tidak dimodifikasi secara ilegal dan membungkus logika internal agar lebih rapi.
   - Implementasi: Menggunakan akses modifier private pada setiap atribut, serta menyediakan method public berupa Getter (seperti getNim()) dan Setter (seperti setStatus()) sebagai pintu akses resmi.

B. Bagian 2 - Desain Class Diagram

1. Buat class diagram sederhana berdasarkan analisis sistem yang telah dibuat. Ketentuan:
   - Minimal 3 class
   - Menunjukkan hubungan antar class



C. Bagian 3 – Analisis Konsep PBO

Jawablah pertanyaan berikut:

1. Jelaskan dimana konsep class dan object diterapkan dalam program Anda.
   Jawab :
   a. Class: Berperan sebagai *blueprint* (cetakan), yaitu Mahasiswa, Perusahaan, Magang, dan FormulirPendaftaran yang mendefinisikan atribut serta *method*.
   b. Object: Perwujudan nyata dari *class*.

      - Penerapan: Saat eksekusi new Mahasiswa(...), program membuat objek spesifik berdasarkan input user yang kemudian disimpan ke dalam ArrayList.

2. Jelaskan bagaimana program Anda menerapkan enkapsulasi.
   Jawab :

   a. Penerapan Enkapsulasi

Program melindungi integritas data dari akses langsung pihak luar melalui dua mekanisme:

- Akses Modifier Private: Mengunci semua atribut (seperti nim dan status) agar tidak bisa dimodifikasi secara langsung oleh kelas Main.
- Getter dan Setter: Menyediakan metode publik (getNim(), setStatus()) sebagai gerbang resmi untuk membaca dan memperbarui data secara terkontrol.

3. Mengapa pendekatan PBO lebih baik dibandingkan pendekatan prosedural pada sistem yang Anda buat?
   Jawab :
   Pendekatan PBO lebih unggul untuk sistem pendaftaran ini karena:

   - Modularitas: Data dibungkus rapi dalam objek (Mahasiswa, Perusahaan, Magang), sehingga kode tidak berantakan dan lebih mudah dibaca dibanding variabel terpisah pada prosedural.
   - Maintenance: Perubahan struktur data (misal: tambah nomor telepon) hanya perlu dilakukan pada kelas terkait tanpa mengganggu logika utama program.
   - Reusability: Melalui *Inheritance*, FormulirPendaftaran menggunakan ulang kode dari Mahasiswa, sehingga menghindari duplikasi penulisan atribut.
   - Keamanan Data: Enkapsulasi memungkinkan adanya validasi (misal: durasi minimal 1 bulan) melalui *setter*, yang sulit dikontrol ketat pada pendekatan prosedural.

D. Bagian 4 – Refleksi

1. Bagian yang paling sulit dalam mengerjakan tugas ini.
   Jawab:
   Bagian tersulit adalah mengintegrasikan hubungan antar class (Inheritance dan Composition) agar data dari objek Mahasiswa, Perusahaan, dan Magang dapat tersimpan dan ditampilkan secara sinkron dalam satu objek FormulirPendaftaran, terutama saat melakukan pembaruan status berdasarkan NIM di dalam ArrayList.

2. Hal baru yang Anda pelajari tentang konsep PBO.
   Jawab:
   - Pada konsep "Warisan" (Inheritance) Saya baru paham fungsinya extends. Saya tidak perlu mengetik ulang variabel NIM atau Nama di dalam FormulirPendaftaran karena sudah "warisan" dari Mahasiswa.
   - Ternyata sebuah Class (seperti FormulirPendaftaran) bisa menyimpan objek utuh dari Class lain (Perusahaan dan Magang). Ini sangat memudahkan karena data jadi terkelompok rapi sesuai fungsinya, persis seperti di dunia nyata di mana sebuah formulir fisik memang berisi informasi dari pihak yang berbeda-beda.

3. Jika sistem ini dikembangkan lebih lanjut, fitur apa yang akan ditambahkan?
   Jawab:

   - Validasi Input: Menambahkan pengecekan agar durasi magang tidak bernilai negatif dan NIM harus angka.
   - Penyimpanan Permanen (Database/File): Menambahkan fitur simpan ke file .txt atau database agar data tidak hilang saat program ditutup.
   - Fitur Login: Memisahkan hak akses antara Mahasiswa (hanya bisa daftar) dan Admin (bisa update status).